

INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO YAVIRAC



CARRERA DE DESARROLLO DE SOFTWARE

TEMA

5ªA Trabajo Colaborativo en Parejas

Quinto “A” Matutino

AUTORES: Latorre Andrade Ronald Israel

: Vera Bravo Angelo Joel

TUTOR:

Byron Moreno

QUITO – ECUADOR

2026 - I



1. INVESTIGACIÓN TEÓRICA

Control de versiones:

Definición:

Es un sistema que registra meticulosamente los cambios realizados sobre un archivo o conjunto de datos a lo largo del tiempo, permitiendo recuperar versiones específicas del historial cuando sea requerido.

Utilidad: Facilita el trabajo simultáneo de múltiples programadores sobre una misma base de código, previniendo la pérdida de información y proporcionando un respaldo histórico de todo el desarrollo.

Ejemplo práctico: Si una nueva actualización genera un fallo crítico en el sistema, el control de versiones permite revertir el estado del proyecto al punto exacto anterior donde funcionaba correctamente. Fuente bibliográfica: Chacon, S., & Straub, B. (2014). Pro Git. Apress.

Git: Definición: Es un software de control de versiones distribuido, de código abierto y gratuito, diseñado para gestionar el historial de desarrollo de proyectos informáticos con velocidad y eficiencia. Utilidad: Guarda el historial de forma local en la máquina de cada desarrollador, permitiendo realizar operaciones de guardado de manera independiente y sin necesidad de conexión constante a la red.

Ejemplo práctico: Inicializar el rastreo de una carpeta local mediante la ejecución del comando "git init" en la consola de comandos.

Fuente bibliográfica: Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media.

GitHub: Definición: Es una plataforma de alojamiento web y computación en la nube utilizada para centralizar y respaldar repositorios de código que emplean el sistema Git.

Utilidad: Proporciona herramientas web colaborativas esenciales, tales como control de accesos, gestión de Pull Requests, seguimiento de errores e interfaces gráficas para la administración de equipos.

Ejemplo práctico: Alojar el código del proyecto escolar en una URL pública de internet para compartir el progreso directamente con el docente examinador. Fuente bibliográfica: Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media.

Repositorio local:

Definición: Es el contenedor de archivos y metadatos históricos administrados por Git que reside de forma física y privada en el disco duro del ordenador del programador.

Utilidad: Permite guardar progresos intermedios y realizar ramas experimentales de forma ágil sin alterar el servidor central del equipo.

Ejemplo práctico: La carpeta de desarrollo ubicada en el directorio de la computadora del estudiante, la cual incluye el directorio oculto ".git".
Fuente bibliográfica: Git SCM Documentation. (2026). Git Basics - Getting a Git Repository. Recuperado de git-scm.com.

Commit :

Definición: Es una confirmación o instantánea registrada de forma permanente en el historial de Git que representa el estado del código en Concepto Definición, Utilidad, Ejemplo y Bibliografía un instante específico. **Utilidad:**

Establece puntos de control seguros a los cuales se puede regresar de forma exacta en caso de fallas posteriores.

Ejemplo práctico: Guardar los cambios finales de un archivo de texto ejecutando el comando 'git commit -m "feat: agregar archivo inicial holamundo"'.
Fuente bibliográfica: Pilgrim, M. (2010). Dive Into HTML5. O'Reilly Media.

Branch (Rama)

Definición: Es una bifurcación o línea de tiempo independiente de desarrollo que permite realizar modificaciones en el código sin interferir con otras líneas de trabajo.

Utilidad: Permite a los desarrolladores aislar el código inestable o las nuevas funcionalidades de la versión de producción o de la rama principal limpia.

Ejemplo práctico: Desarrollar en la rama "rama-estudiante-b" de forma remota mientras la rama principal se mantiene estable. **Fuente bibliográfica:**

Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media. **Merge**

Definición: Es la acción de fusionar o unificar dos o más ramas de Git para consolidar sus respectivos historiales y líneas de código en un solo flujo común.

Utilidad: Integra las características desarrolladas de manera aislada dentro del núcleo o tronco central del proyecto una vez que han sido probadas.

Ejemplo práctico: Fusionar los cambios terminados de "rama-estudiante-b" dentro de la rama principal mediante la consola de comandos o la web de GitHub.

Fuente bibliográfica: Chacon, S., & Straub, B. (2014). Pro Git. Apress.

Pull Request (PR)

Definición: Es una solicitud en plataformas como GitHub para proponer la incorporación de los cambios realizados en una rama de trabajo hacia la rama principal del proyecto.

Utilidad: Facilita la revisión por pares, habilitando un espacio interactivo de discusión, control de calidad y sugerencias antes de alterar el código base.

Ejemplo práctico: Levantar un Pull Request en GitHub para que el compañero de equipo audite el mensaje del archivo Hola Mundo. Concepto Definición, Utilidad, Ejemplo y Bibliografía

Fuente bibliográfica: GitHub Docs. (2026). About pull requests. Recuperado de docs.github.com. **Fork**

Definición: Es una de las características de GitHub que genera una copia exacta de un repositorio perteneciente a otra cuenta directamente dentro del espacio del usuario que lo solicita.

Utilidad: Permite realizar modificaciones pesadas o proponer correcciones a proyectos ajenos o de código abierto sin contar con accesos administrativos directos al original.

Ejemplo práctico: Copiar el repositorio público de una librería famosa a una cuenta personal para estudiar su estructura internamente.

Fuente bibliográfica: Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media.

Conflictos de integración

Definición: Es el estado de bloqueo que se produce cuando Git no puede resolver de forma automática la unión de dos cambios concurrentes aplicados sobre la misma sección o línea de un archivo.

Utilidad: Detiene el flujo de integración para requerir la intervención humana, previniendo de esta forma que un cambio destruya involuntariamente el trabajo de otro desarrollador. **Ejemplo práctico:**

Intentar hacer la fusión de dos ramas que editaron con textos distintos la línea número 2 del archivo "holamundo.txt".



Fuente bibliográfica: Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media.

Integración Continua (CI)

Definición: Es la práctica de desarrollo que consiste en unificar los cambios de código en la rama principal de manera frecuente, disparando ejecuciones y pruebas automáticas en cada iteración.

Utilidad: Identifica de manera inmediata fallos de compilación y errores lógicos, acortando los tiempos de respuesta ante fallas técnicas.

Ejemplo práctico: Configurar una herramienta que compile el proyecto e inspeccione la sintaxis cada vez que un miembro del equipo realice un push.

Fuente bibliográfica: Fowler, M. (2006). Continuous Integration. MartinFowler.com.

Entrega Continua (CD)

Definición: Práctica donde el código validado que ha superado las etapas previas de Integración Continua es compilado y empaquetado. Concepto Definición, Utilidad, Ejemplo y Bibliografía automáticamente, manteniéndolo listo para producción.

Utilidad: Garantiza que el software se encuentre siempre en un estado desplegable, permitiendo realizar lanzamientos a producción con una decisión humana manual inmediata.

Ejemplo práctico: Compilar automáticamente los instaladores de una aplicación cada noche para que queden disponibles para el equipo de pruebas.

Fuente bibliográfica:

Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley Professional. **Despliegue Continuo**

Definición: Es la automatización avanzada donde cualquier cambio que supera satisfactoriamente el flujo completo de pruebas y validaciones automáticas se publica directo en producción sin intervención humana.

Utilidad: Minimiza los tiempos de espera operativos, permitiendo que las mejoras de software e innovaciones lleguen de inmediato a los clientes finales.

Ejemplo práctico:

Subir un cambio de estilo a la rama central y observar que la página web oficial en internet se actualiza de manera autónoma en pocos minutos.



Fuente bibliográfica: Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley Professional.

GitHub Actions

Definición: Es un servicio integrado dentro del ecosistema de GitHub que permite automatizar, personalizar y ejecutar flujos de trabajo de desarrollo de software directamente desde el repositorio mediante archivos YAML.

Utilidad: Provee la infraestructura idónea para programar sistemas completos de Integración Continua y Despliegue Continuo (CI/CD) sin recurrir a servidores externos.

Ejemplo práctico: Escribir el archivo "verificacion.yml" para automatizar un conjunto de tareas cada vez que se detecte una solicitud de extracción (PR).

Fuente bibliográfica: GitHub Docs. (2026). Understanding GitHub Actions. Recuperado de docs.github.com.

Paso uno:

Creación del repositorio en GitHub El Estudiante A (ronalfcito) procedió a la configuración inicial y creación del repositorio público en GitHub con el nombre "practicaGit", cumpliendo con la visibilidad solicitada por la guía de la práctica.

Figura 1: Interfaz web de GitHub que documenta la creación exitosa del repositorio público. (El Estudiante A)

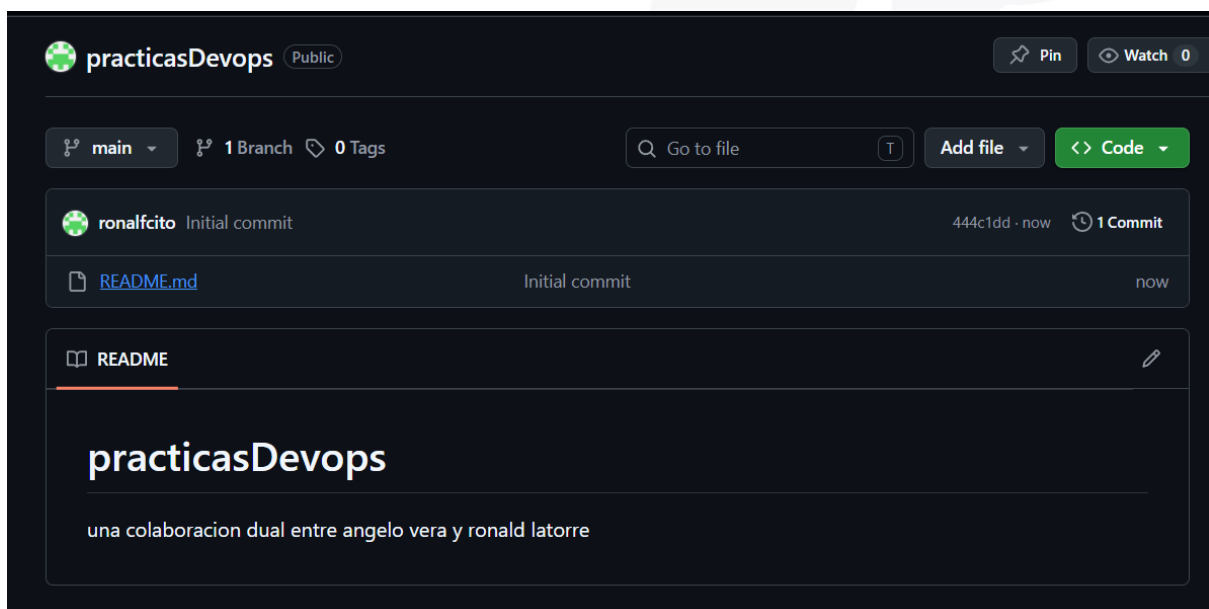


Figura 1.1: Estuante b (Ajvera2000)

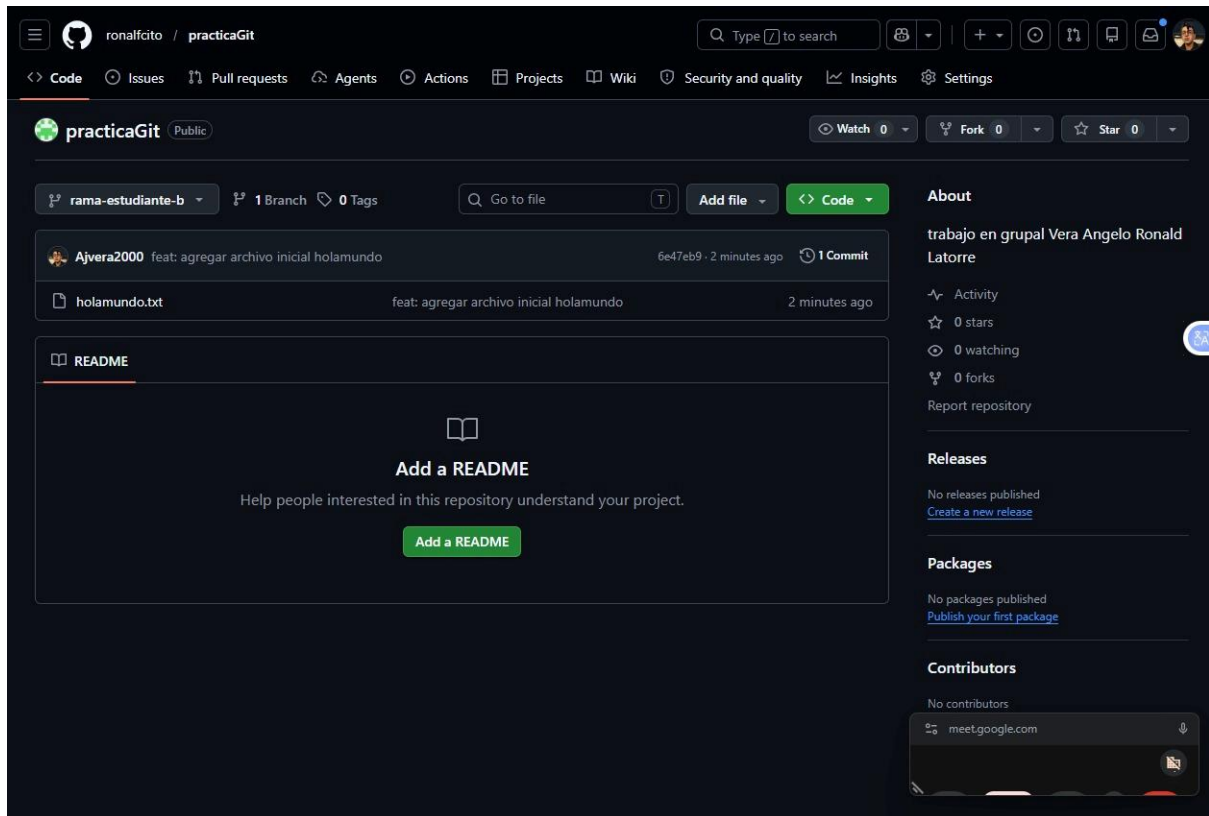


Figura 1.2 invitación a el repositorio de GitHub. (El Estudiante A)

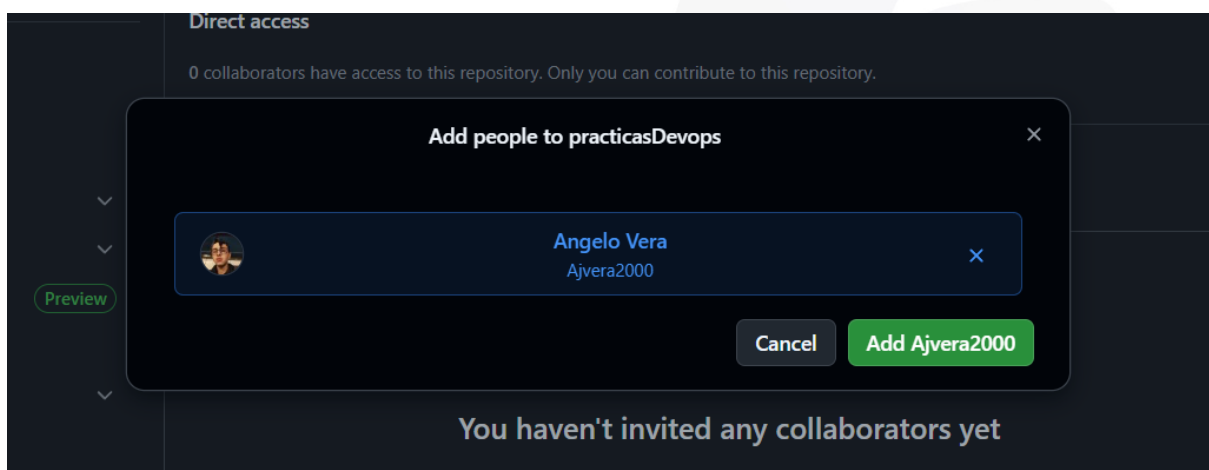
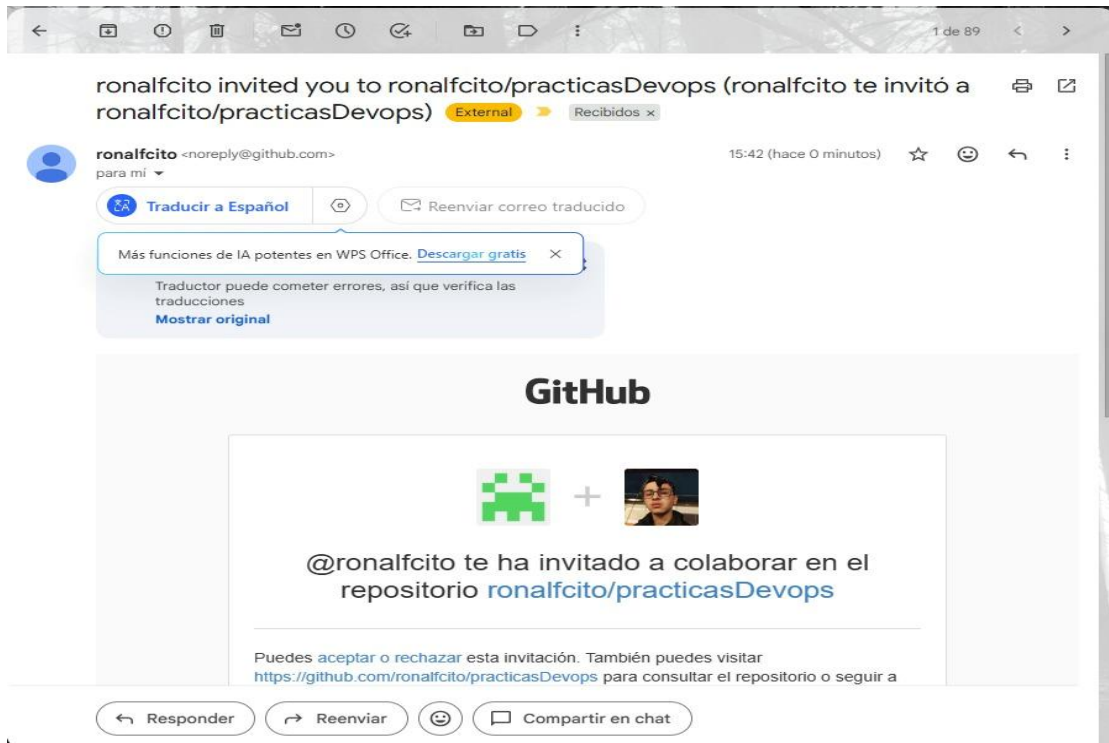


Figura 1.3 invitación a el repositorio de GitHub. (El Estudiante B)



Paso dos:

Creación de ramas de trabajo por cada integrante:

Para el flujo inicial, el Estudiante B (Ajvera2000) generó y se posicionó en una nueva bifurcación utilizando el comando 'git checkout -b rama-estudiante-b'. Como el proyecto se inició completamente vacío, esta pasó a registrarse en el selector web de GitHub.

Figura 2: (El Estudiante B)

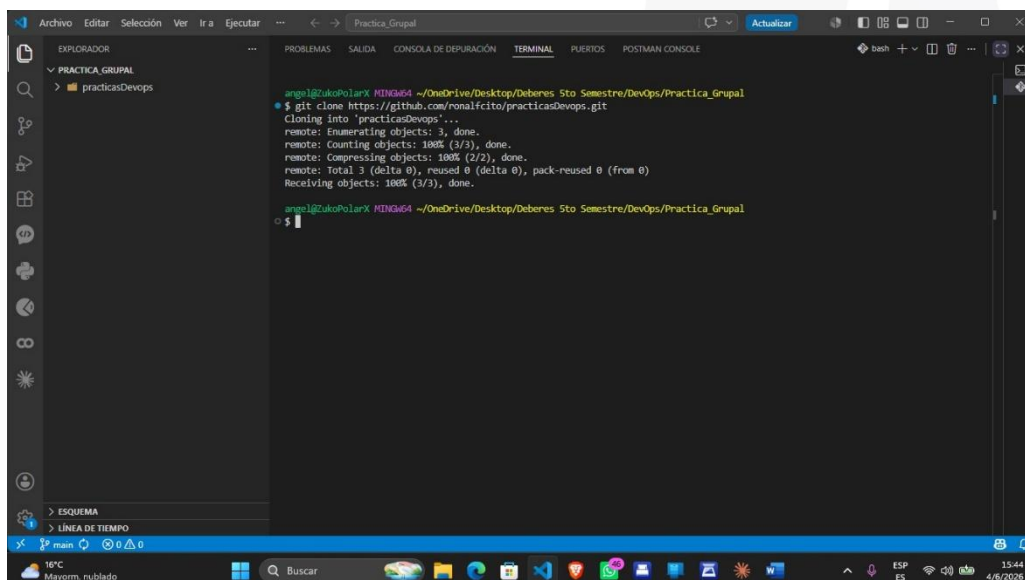
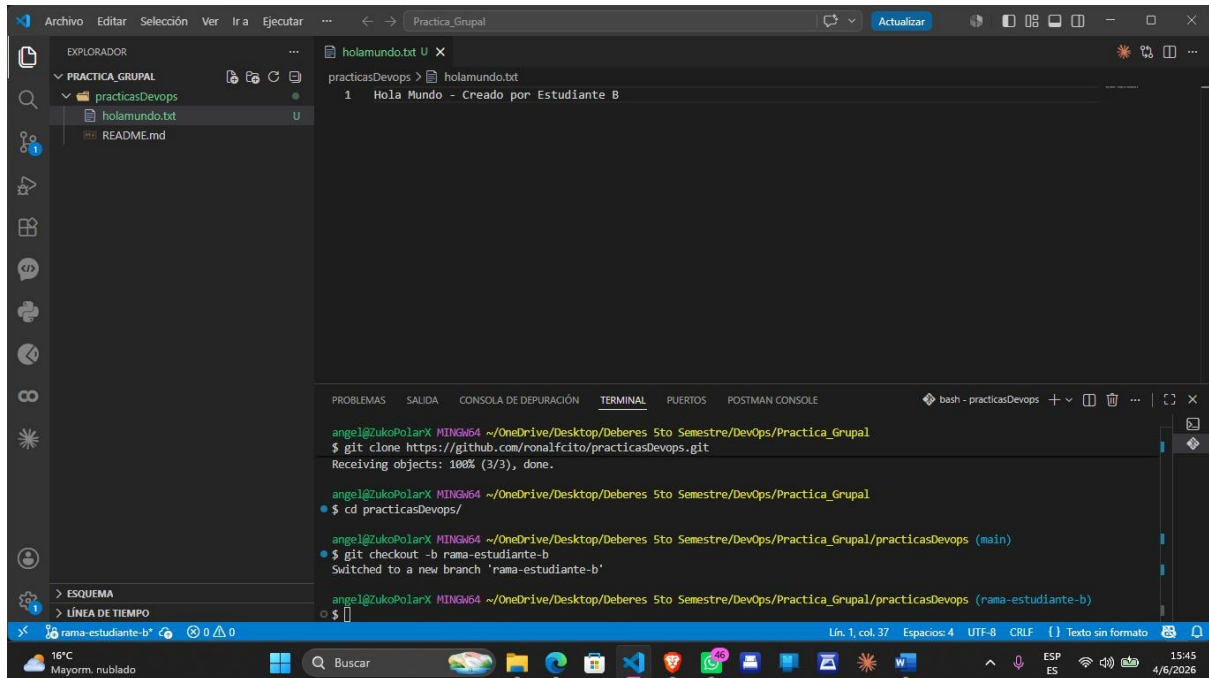


Figura 2.1: Menú de selección de ramas de GitHub constatando la presencia de la rama de trabajo independiente. (El Estudiante B)

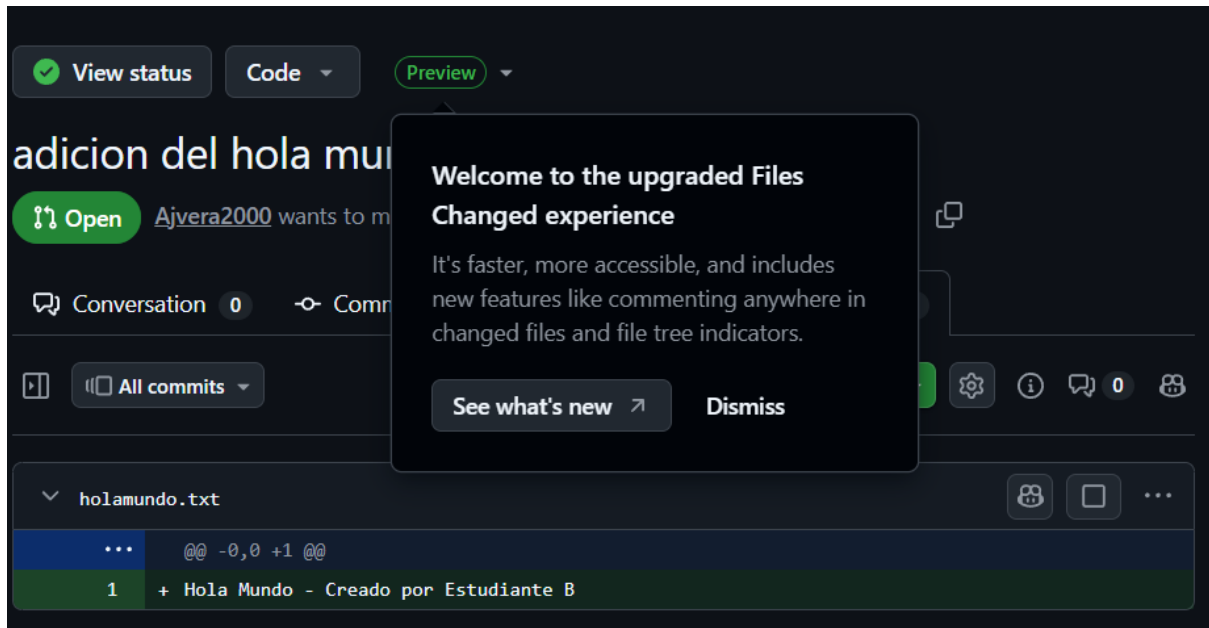


Paso 3

Commits realizados por ambos integrantes

Ambos integrantes guardaron sus respectivos avances de forma local. A continuación se evidencia el proceso de registro de instantáneas con mensajes descriptivos.

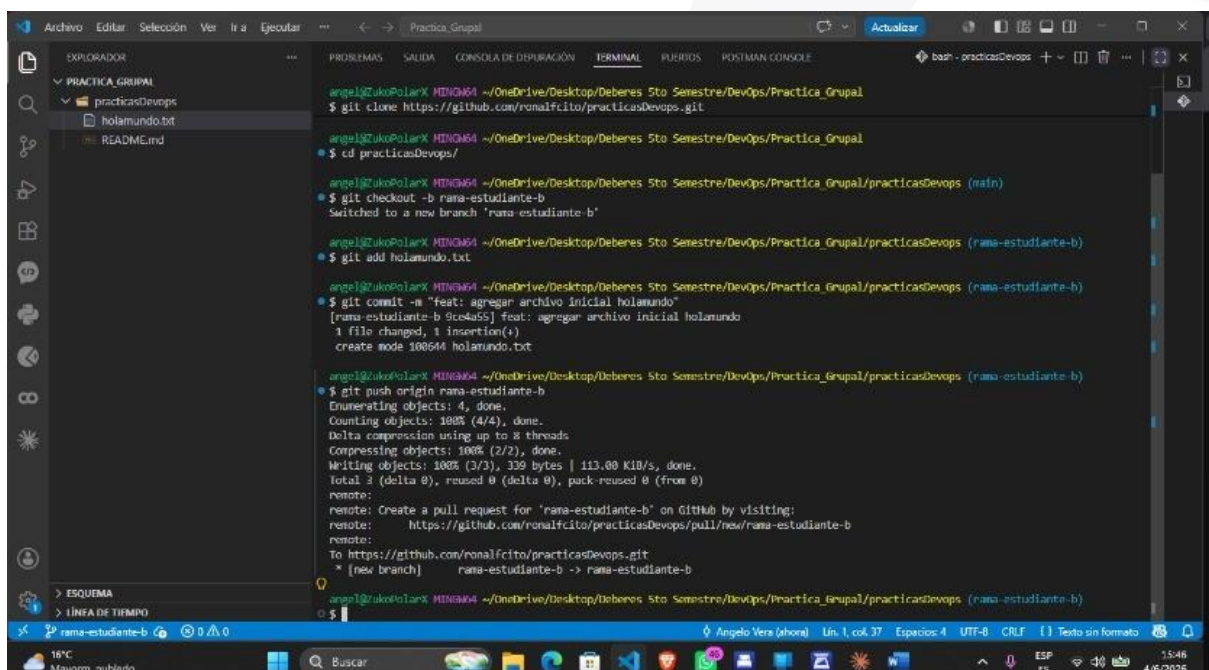
Figura 3: Historial de confirmaciones locales utilizando terminología estandarizada. (El Estudiante A)



Publicación de cambios al repositorio remoto

Las estaciones de trabajo locales fueron sincronizadas con los servidores web de GitHub haciendo uso del comando de publicación hacia el origen remoto: 'git push origin rama-estudiante-b'.

Figura 3.1: Evidencia del proceso de transmisión de datos locales hacia el repositorio de GitHub. (El Estudiante B)

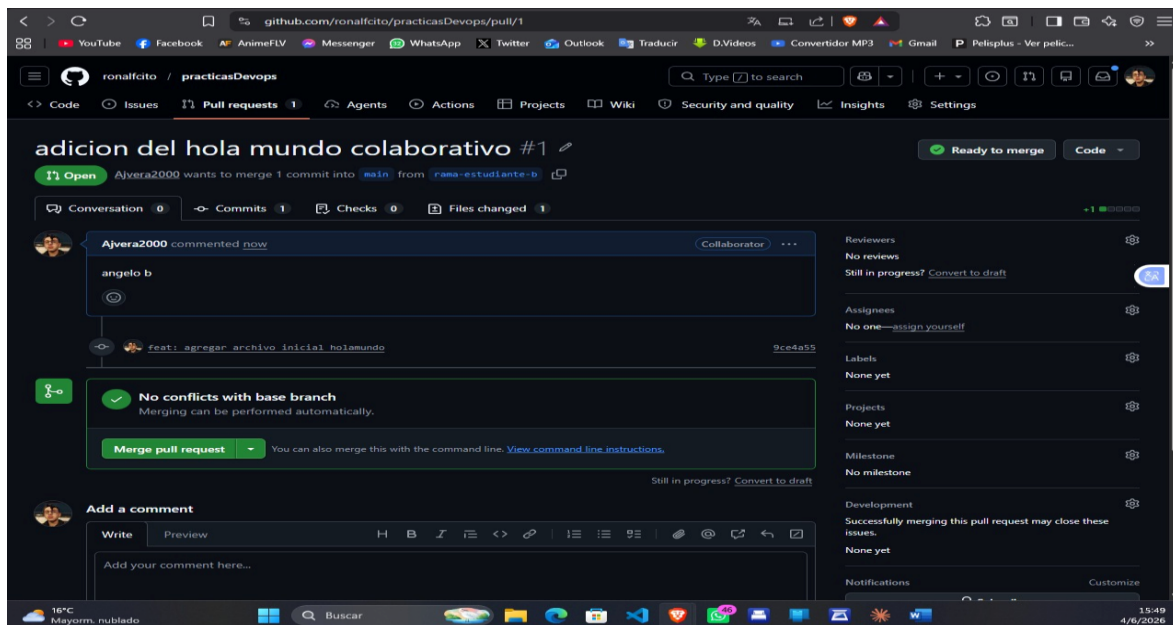


Creación de Pull Requests:



El Estudiante B, mediante la interfaz del repositorio, abrió una solicitud de extracción formal para proponer la integración del archivo "holamundo.txt" que contenía el mensaje inicial del proyecto.

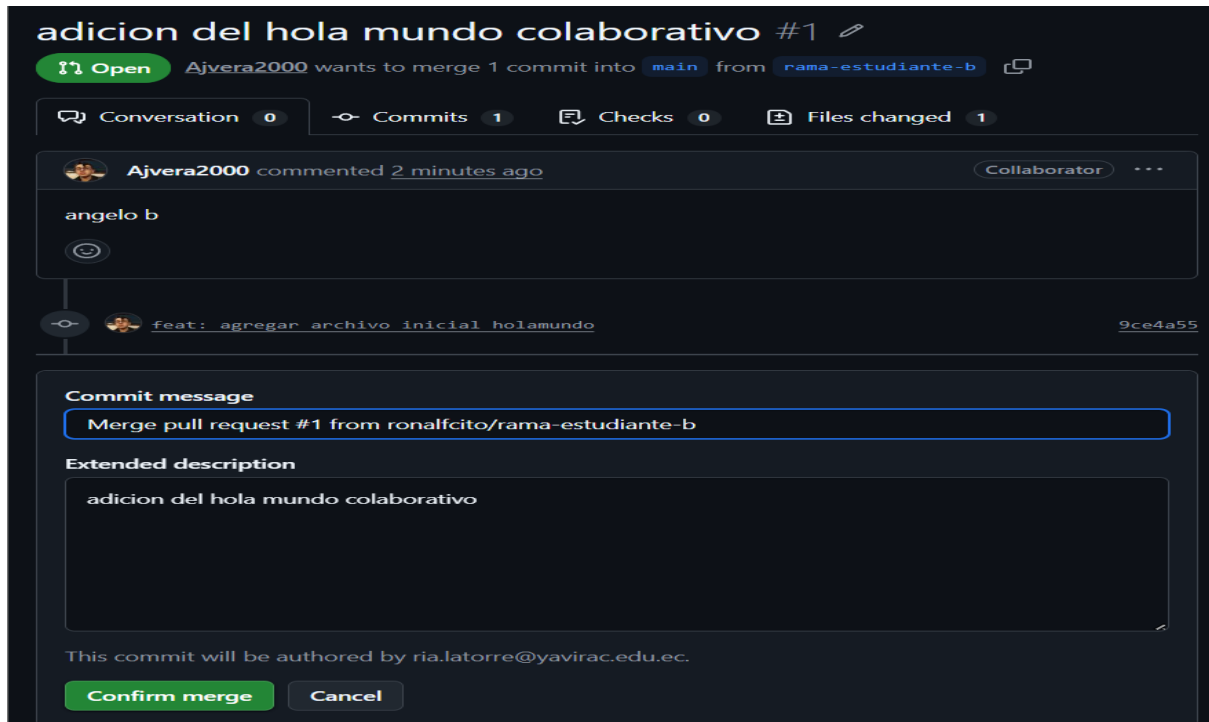
Figura 3.2: Solicitud de extracción (Pull Request) generada para la revisión grupal de cambios.(El Estudiante B)



Revisión de código y aprobación de Pull Requests

El Estudiante A ingresó al módulo de Pull Requests, auditó las líneas modificadas en la pestaña correspondientes y completó la validación técnica del código introducido por su compañero.

Figura 3.3: Panel administrativo de GitHub donde se documenta la aprobación por pares.(El Estudiante A)

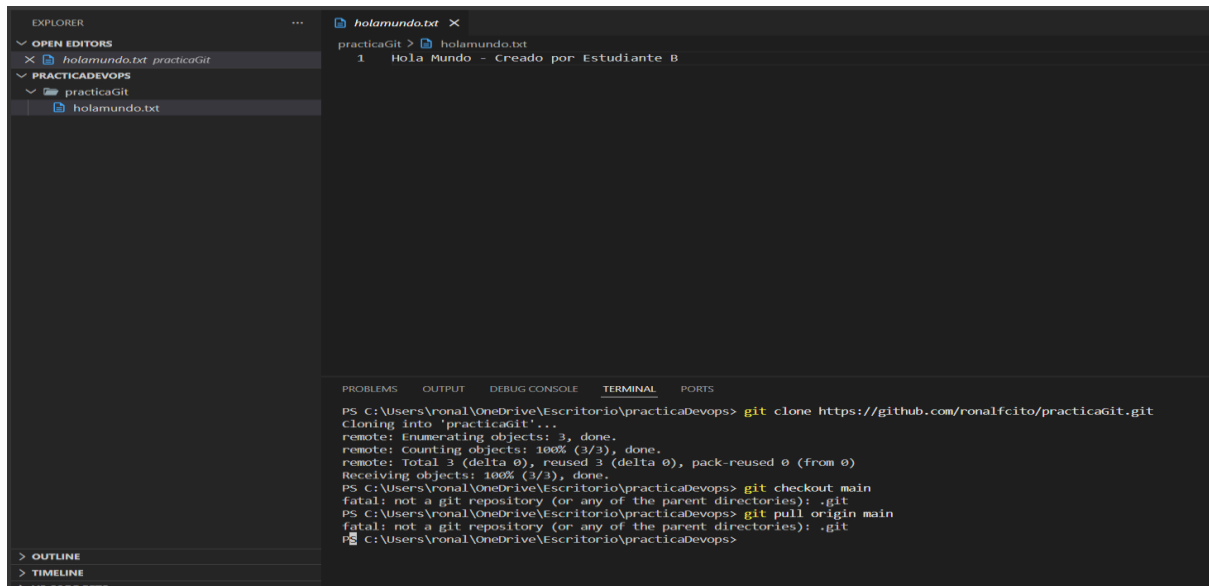


Paso 4:

Intercambio de Roles y Generación del Conflicto

Ahora invertiremos los papeles. El **Estudiante A** desarrollará y el **Estudiante B** revisará. Además, provocaremos un conflicto editando **la misma línea del mismo archivo**.

Figura 4: Antes de programar, el **Estudiante A** debe traer los cambios que acaba de fusionar en la nube:



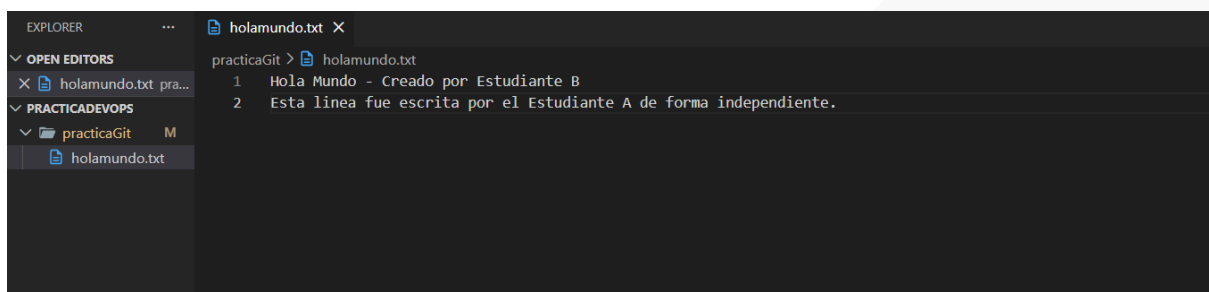
```

EXPLORER
  OPEN EDITORS
    holamundo.txt practicaGit
  PRACTICADEVOPS
    practicaGit
    holamundo.txt

practicaGit > holamundo.txt
1 Hola Mundo - Creado por Estudiante B

PS C:\Users\ronal\OneDrive\Escritorio\practicaDevops> git clone https://github.com/ronalfcito/practicaGit.git
Cloning into 'practicaGit'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
PS C:\Users\ronal\OneDrive\Escritorio\practicaDevops> git checkout main
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\ronal\OneDrive\Escritorio\practicaDevops> git pull origin main
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\ronal\OneDrive\Escritorio\practicaDevops>
  
```

Figura 4.1: Abre holamundo.txt, se ubica en la línea 2 y escribe: (El Estudiante A)

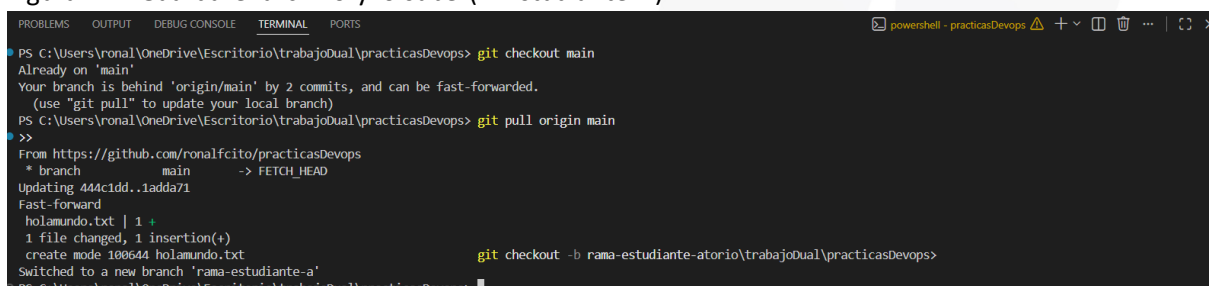


```

EXPLORER
  OPEN EDITORS
    holamundo.txt pra...
  PRACTICADEVOPS
    practicaGit M
    holamundo.txt

practicaGit > holamundo.txt
1 Hola Mundo - Creado por Estudiante B
2 Esta línea fue escrita por el Estudiante A de forma independiente.
  
```

Figura 4.2: Guarda el archivo y lo sube (El Estudiante A)



```

PS C:\Users\ronal\OneDrive\Escritorio\trabajoDual\practicasDevops> git checkout main
Already on 'main'
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
(Use "git pull" to update your local branch)
PS C:\Users\ronal\OneDrive\Escritorio\trabajoDual\practicasDevops> git pull origin main
>>
From https://github.com/ronalfcito/practicasDevops
* branch      main      -> FETCH_HEAD
Updating 444c1dd..1adda71
Fast-forward
 holamundo.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 holamundo.txt
Switched to a new branch 'rama-estudiante-atorio\trabajoDual\practicasDevops'
PS C:\Users\ronal\OneDrive\Escritorio\trabajoDual\practicasDevops>
  
```

Figura 4.3: Estudiante B crea OTRA rama y modifica la línea 2 al mismo tiempo

Estudiante B (sin saber lo que hizo A) va a su terminal, regresa a main para estar al día y crea una nueva rama:

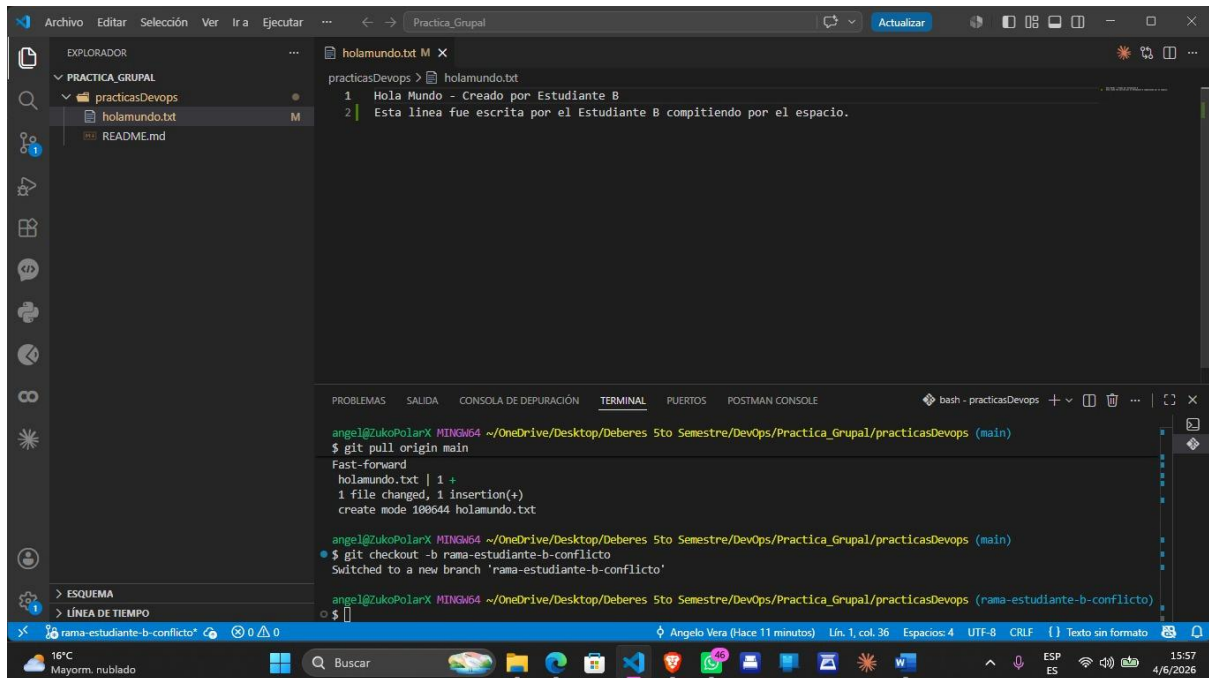
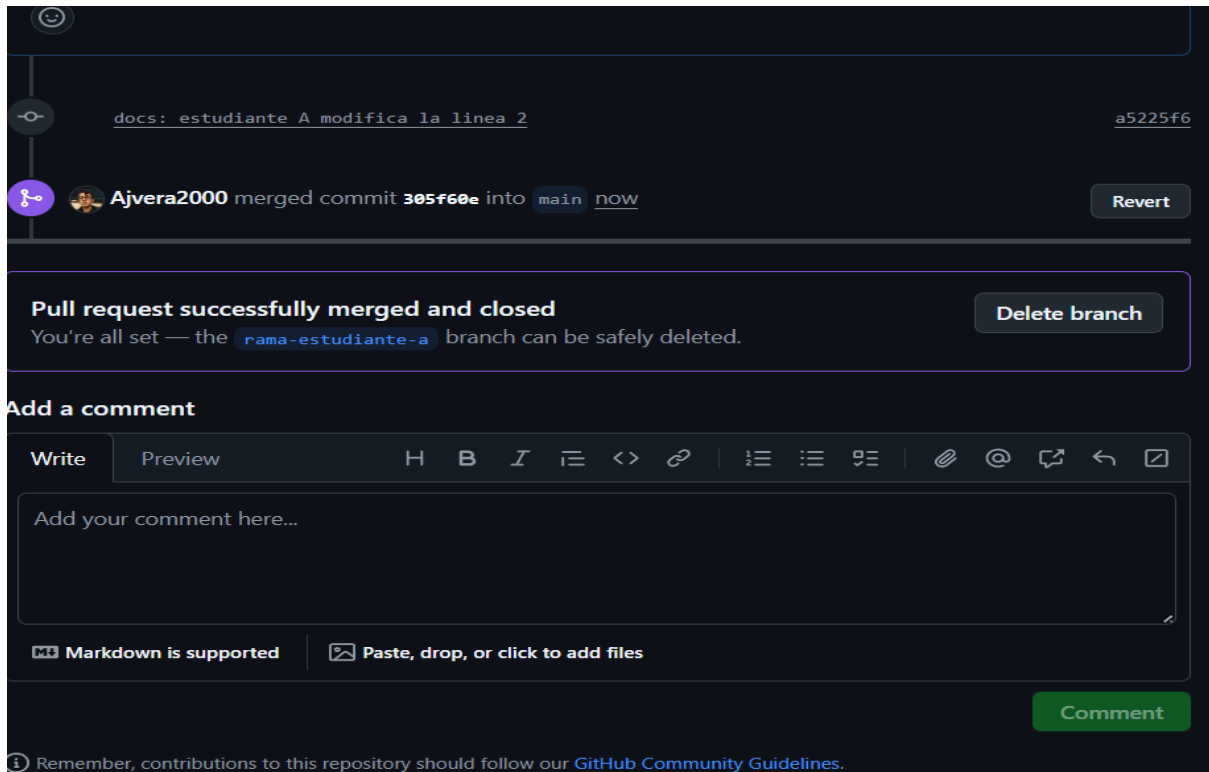


Figura 4.3: El choque en GitHub(El Estudiante B)



Figura 4.4:GitHub bloqueará el botón verde y mostrará un mensaje en gris/rojo(El Estudiante A)



Paso 5

Figura 5.1: Bloqueo de seguridad provocado por la colisión de dos ediciones en la misma línea. (El Estudiante A)

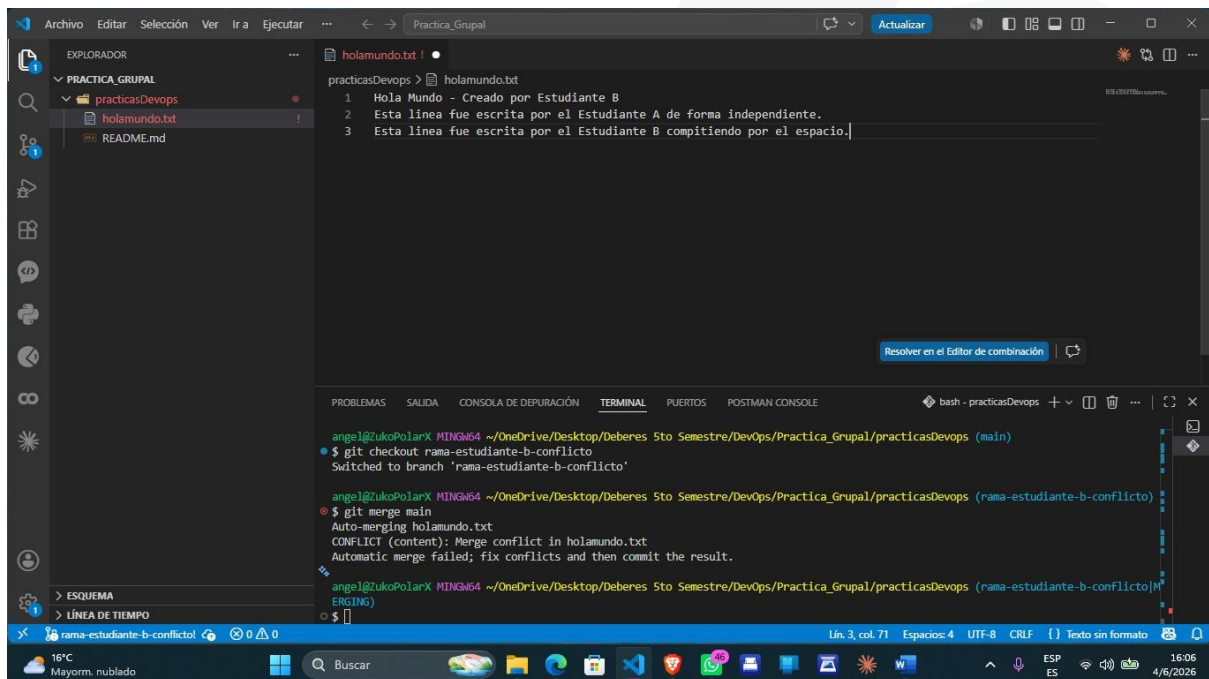


Figura 5.2: Marcadores de conflicto desplegados dentro de Visual Studio Code previo a la unificación manual.(El Estudiante B)

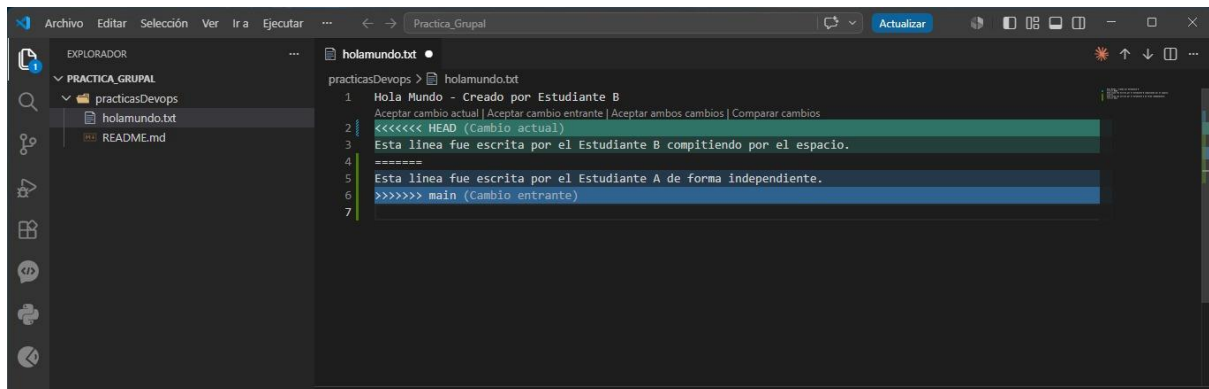
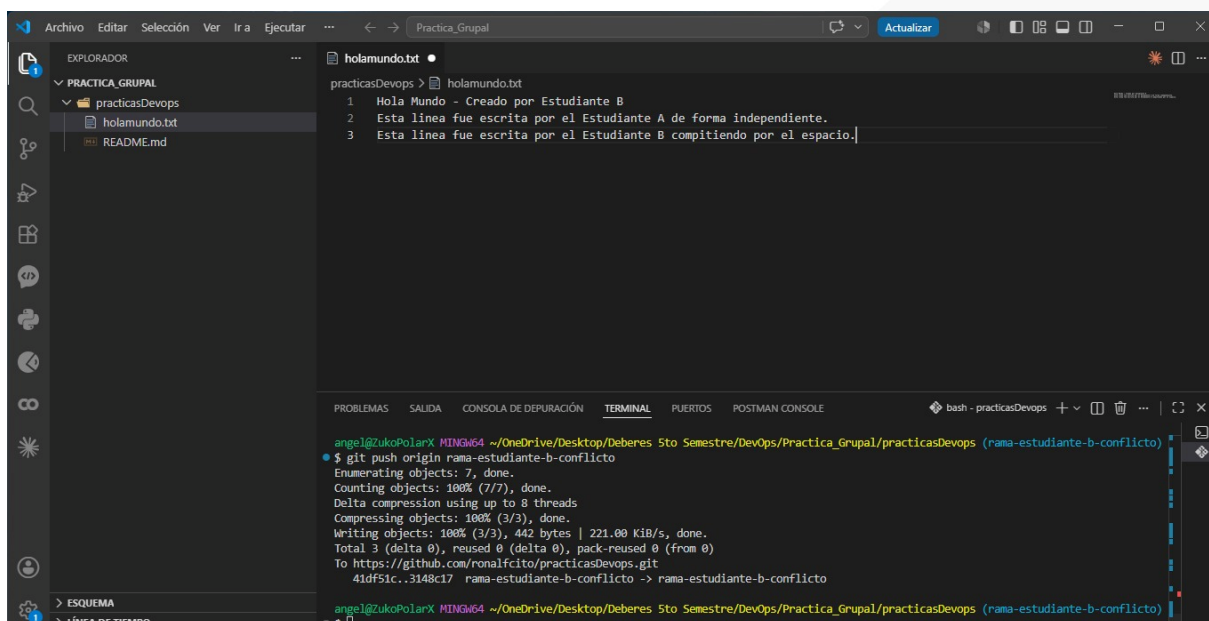
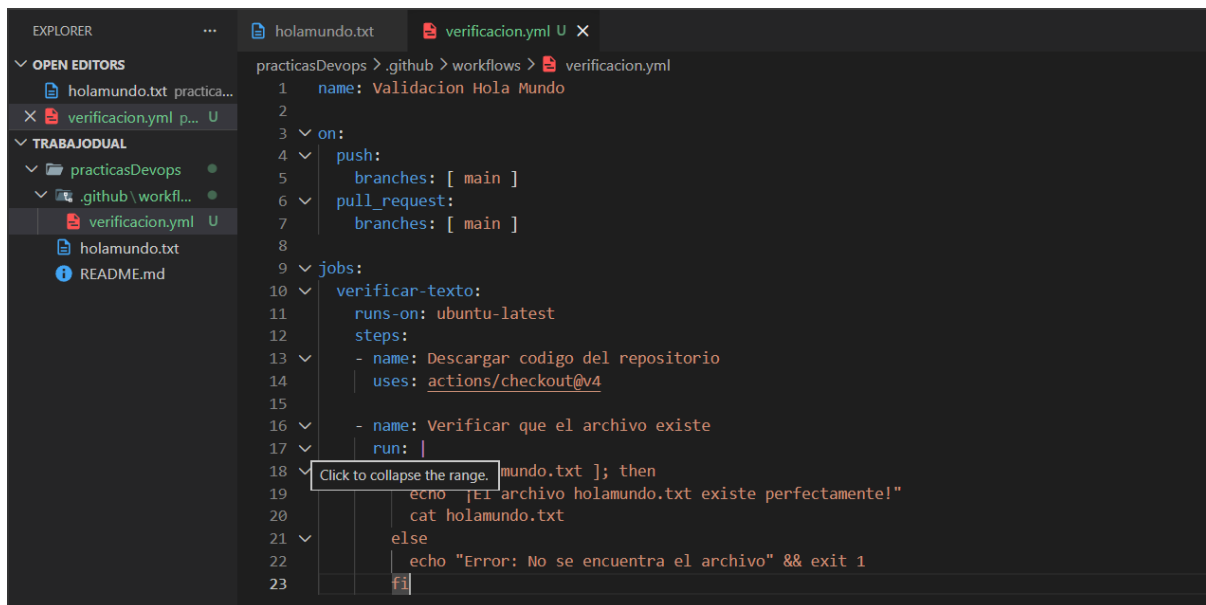


Figura 5.3. El Estudiante B resolvió la incidencia localmente trayendo main actualizado y ejecutando 'git merge main'. El conflicto se abrió en el editor de código mostrando los delimitadores nativos de Git antes de ser depurados y corregidos de mutuo acuerdo.(El Estudiante B)



Ejecución de GitHub Actions:

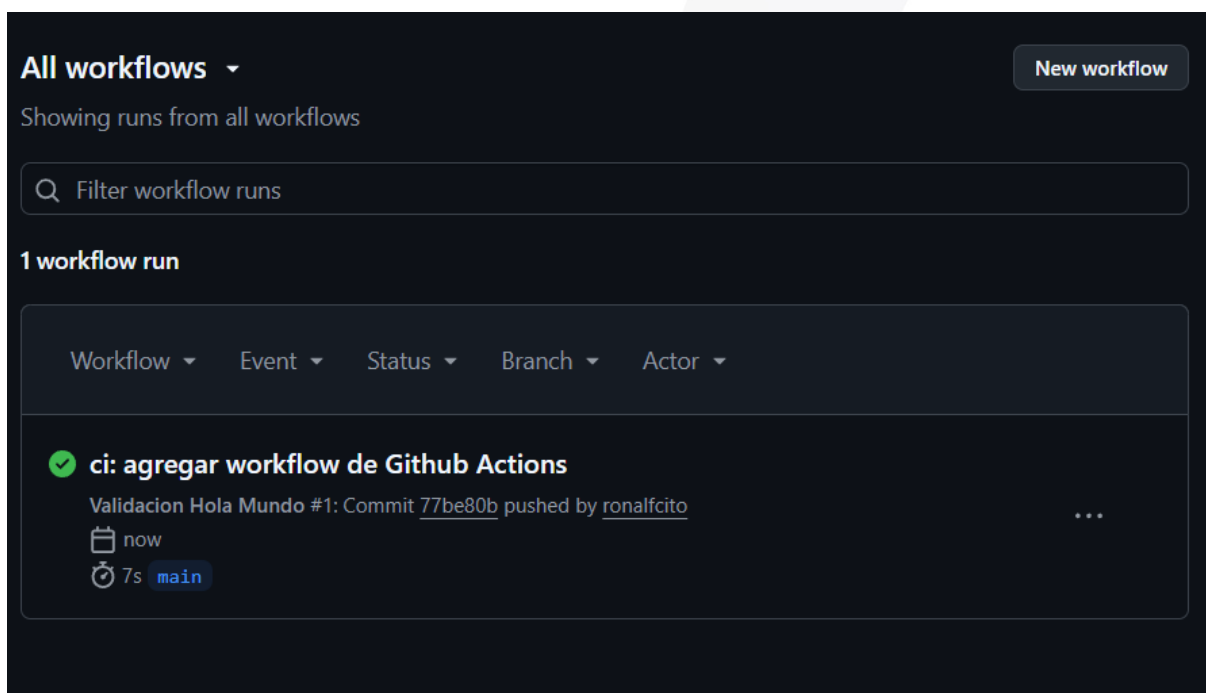
Figura 6.1 Se incorporó el flujo automatizado mediante el archivo de configuración ".github/workflows/verificacion.yml". Tras corregir el conflicto e integrar la solución, se disparó la ejecución automatizada en los servidores de la plataforma, concluyendo con éxito. (El Estudiante A)



```

1  name: Validacion Hola Mundo
2
3  on:
4    push:
5      branches: [ main ]
6    pull_request:
7      branches: [ main ]
8
9  jobs:
10   verificar-texto:
11     runs-on: ubuntu-latest
12     steps:
13       - name: Descargar codigo del repositorio
14         uses: actions/checkout@v4
15
16       - name: Verificar que el archivo existe
17         run: |
18           Click to collapse the range. mundo.txt ]; then
19           echo "El archivo holamundo.txt existe perfectamente!"
20           cat holamundo.txt
21         else
22           echo "Error: No se encuentra el archivo" && exit 1
23         fi
  
```

Figura 6.2: Pestaña 'Actions' exhibiendo el visto verde de cumplimiento del flujo de Integración Continua. (El Estudiante A)



CONCLUSIONES INDIVIDUALES

Conclusión de Ronald Latorre **(Estudiante A)**: A través de esta práctica colaborativa, logré comprender a fondo la gran responsabilidad que conlleva el rol de revisor y administrador dentro de un proyecto de software. La auditoría minuciosa de código durante las solicitudes de extracción (Pull Requests) demostró ser una fase crítica e indispensable para asegurar la calidad y estabilidad antes de ejecutar el Merge definitivo en la rama principal. Asimismo, la implementación de GitHub Actions me permitió asimilar el valor estratégico de la Integración Continua; contar con flujos automatizados que validen la sintaxis y compilen el código en cada iteración proporciona al equipo la certeza absoluta de que ninguna funcionalidad nueva afectará o romperá las características previamente consolidadas del sistema.

Conclusión de Angelo Joel Vera Bravo **(Estudiante B)**: La realización de este laboratorio me permitió consolidar destrezas técnicas clave para el desarrollo en equipo, especialmente en lo que respecta al aislamiento de tareas mediante la creación y gestión de ramas locales e independientes de trabajo. Experimentar de primera mano la colisión de cambios concurrentes me ayudó a asimilar con claridad la forma exacta en que Git señala y delimita los conflictos utilizando marcadores y símbolos nativos en las líneas afectadas de código. El proceso de solucionar estas incidencias directamente desde Visual Studio Code, contrastando los bloques en disputa y unificando el archivo bajo un consenso mutuo, me demostró que los conflictos de integración no constituyen errores del sistema, sino mecanismos de seguridad esenciales que garantizan la integridad del software y protegen el aporte individual de cada programador.

ENLACE DEL GITHUB:

<https://github.com/ronalfcito/practicasDevops.git>

